



Debre Berhan University

College of Computing

Department

of

Information Technology

Course Title: Advanced Programming

Course Code: ITec 3057

Credit : 3 Cr. Hrs / 5 ECTS (2 Hrs. Lec. & 3 Hrs. Lab)

Target group: 3rd year Information Technology

Course Manager: Yirga G.

Chapter 2:

Multithreading in Java

programming

Introduction to Multithreading

- Executing several tasks simultaneously is the concept of multitasking.
- There are two types of multitasking:
 1. Process based multitasking
 2. Thread based multitasking
- 1. Process based multitasking:
 - Executing several tasks simultaneously where each task is a separate independent program (process) is called process-based multitasking.
 - **Example:** While typing a java program in the editor we can listen audio sounds from same system at the same time we can download a file from Net.
 - All these tasks will be executed simultaneously and independent of each other.
 - Hence it is process-based multitasking.
 - Process based multitasking is best suitable at OS level.

Introduction to Multithreading...

2. Thread based multitasking:

- Executing several tasks simultaneously where each task is a separate independent part of the same program is called thread-based multitasking and each independent part is a thread.
- Thread based multitasking is best suitable at programmatic level.
- ❖ Whether it is process based or thread based, the main objective of multitasking is to reduce response time of the system and to improve performance.

Introduction to Multithreading...

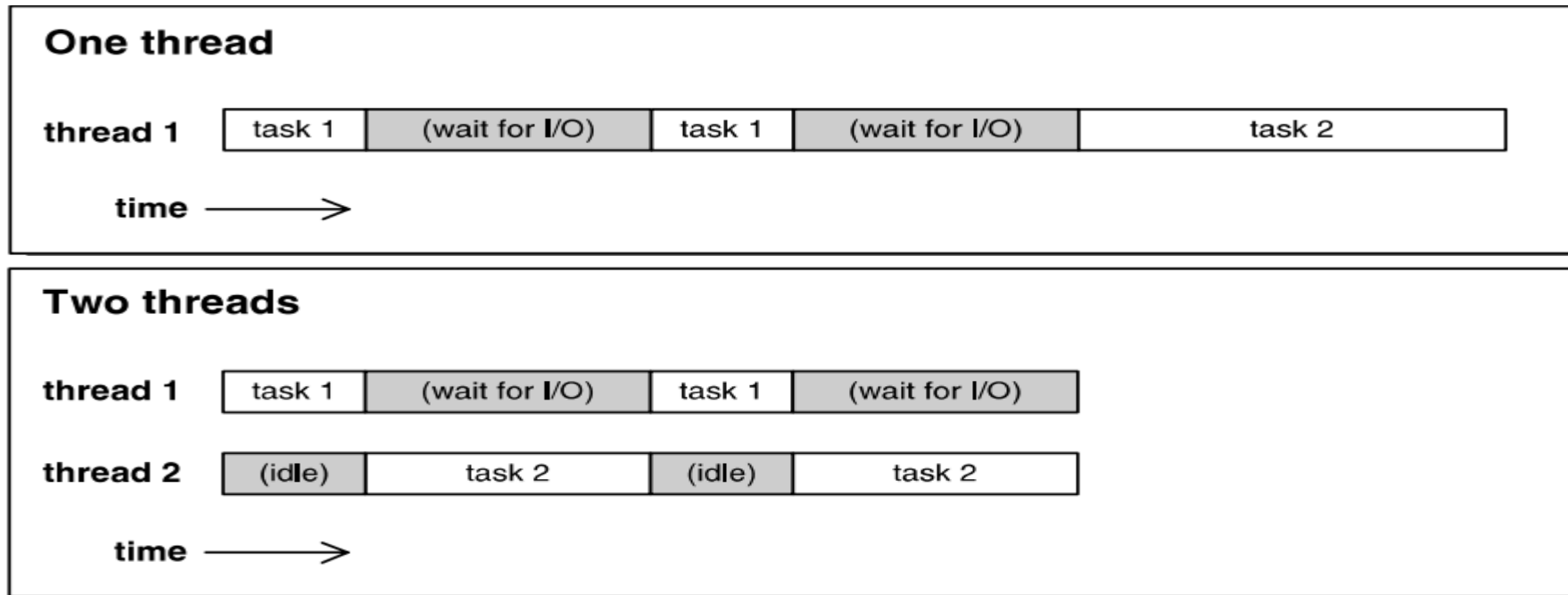
- The main important application areas of multithreading are:
 - To develop multimedia graphics
 - To develop animations
 - To develop video games
 - To develop web and application servers etc.
- When compared with old languages developing multithreaded applications java is very easy because java provides in-built support for multithreading with rich API (Thread, Runnable, ThreadGroup...).

What is Thread?

- Thread is a single sequential flow of control within a program
 - Completes a specific task
- Java application uses a single thread – main
- Java is a multithreaded programming language
- A multithreaded program contains two or more parts that can run concurrently and each part can handle different tasks at the same time
- making optimal use of the available resources
- JVM – thread scheduler, lets portions of each thread execute
- All tasks running at the same time, give efficiency
- Each of the threads can run in parallel

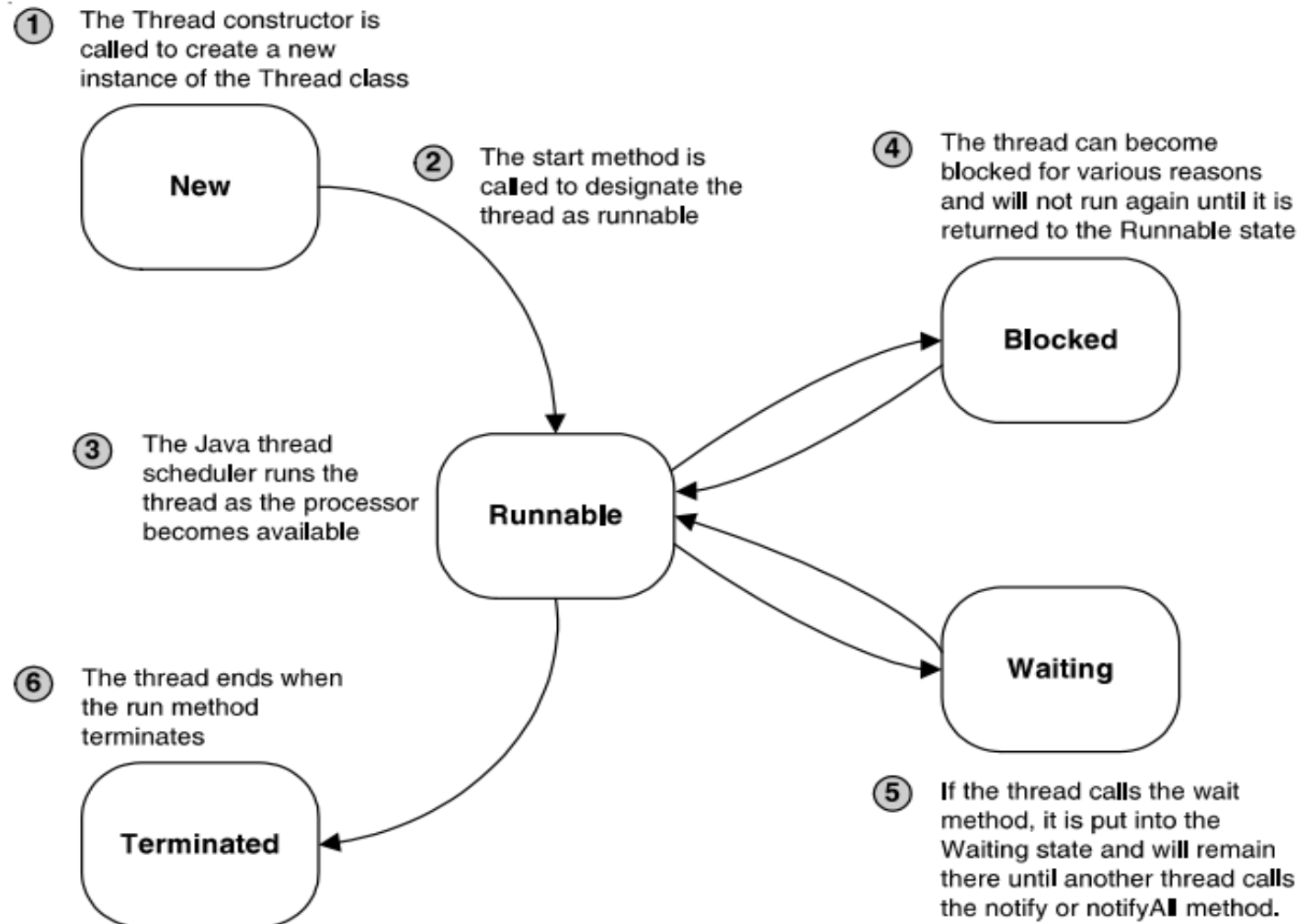
Typical uses of thread

- Improve the applications with extensive I/O operation
- Improve the response of GUI application
- Allow to or more users to run server-based application simultaneously



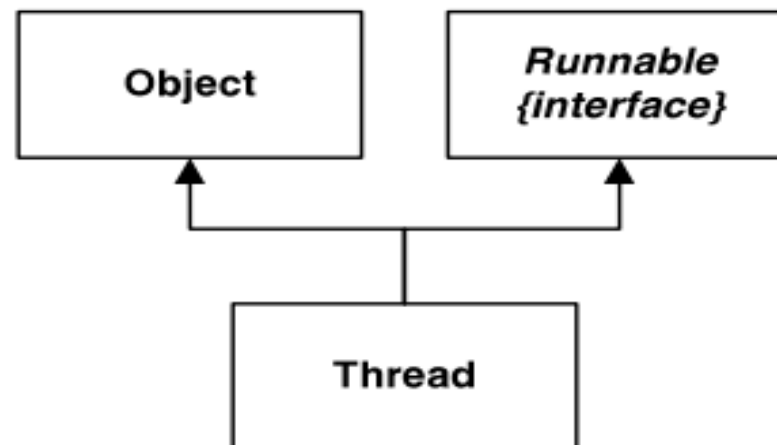
Life Cycle of a Thread

- A thread goes through various stages in its life cycle.
- For example, a thread will be born, started, runs, and then dies



Class and interfaces for working with threads

- Thread class inherits Object class
- Thread class implements Runnable interface
- Runnable interface declares a single method named run
- To **create thread**
 - Define class that **inherits the Thread class**
 - Define a class that **implements the runnable interface**
- The Object class also has some methods that can be used for threading



Methods of Thread class, Runnable interface, and Object class

- **start - Thread**

- Register Thread with the thread scheduler

- **run – Runnable, Thread**

- The thread scheduler calls this methods to run the thread

- **sleep – Thread**

- Causes the current thread to wait (sleep) for a specified period of time so that the CPU can run other threads

- **wait - Object**

- Causes the current thread to wait until another thread calls the notify or notifyAll method for the current object

- **notify - Object**

- Notifies one arbitrary thread that waiting on this object that it can resume

- **notifyAll - Object**

- Notifies all the threads that are waiting on this object that they can resume execution

Contd...

- A thread is a flow of execution where each thread having a separate independent job.

□ How many ways we can define a thread?

- We can define a thread into two ways:
 1. By extending **Thread** class
 2. By implementing **Runnable** interface

Creating a Thread

- The Thread class
 - `java.lang.Thread;`
- Common constructor
 - `Thread()`
 - `Thread(Runnable)`
 - `Thread(String)`
 - `Thread(Runnable, String)`
- Common methods
 - `run()`
 - `start()`
 - `getName()`
 - `currentThread()`
 - `setDaemon(boolean)`
 - `Sleep(long)`
 - `Interrupt()`
 - `isInterrupted()`
 - `yield()`

Creating a Thread

- Create Thread by Implementing Runnable Interface:
 1. Create a class that implements a Runnable interface
 2. Implement a run() method to perform the desired task
`public void run();`
 3. Instantiate a Thread object, supply to Thread constructor
`Thread(Runnable threadObj , String threadName);`
 4. Start by calling start()
`void start();`

Create Thread by Implementing Runnable Interface:

```
public class threadRuunnable {  
    Public static void main(String[] args ){  
        Thread t1 = Thread.currentThread();  
        System.out.println(t1.getName() + " started");  
        Thread t2 = new Thread(IOTask());  
        t2.start();  
        System.out.println(t1.getName() +" starts "+t2.getName());  
        System.out.println(t1.getName() + " finished");  
    }  
}  
  
public class IOTask implements Runnable{  
    @override  
    Public void run(){  
        Thread ct = Thread.currentThread();  
        System.out.println(ct.getName() + " started");  
        Try {  
            Thread.sleep(2000);  
        }  
        catch(InterruptedException e){}  
        System.out.println(ct.getName() + " finished");  
    }  
}
```

Create Thread by Extending Thread Class

1. Create a class that inherits the Thread class
2. Override the run method to perform the desired task
`public void run()`
3. Create the thread by instantiating an object from the class
4. Call the start method
`void start();`

Extending Thread Class ...

```
public class threadExtend {  
    Public static void main(String[] args ){  
        Thread t1 = Thread.currentThread();  
        System.out.println(t1.getName() + " started");  
        Thread t2 = new Thread(IOTask());  
        t2.start();  
        System.out.println(t1.getName() +" starts "+t2.getName());  
        System.out.println(t1.getName() + " finished");  
    }  
}  
  
public class IOTask extends Thread{  
    @override  
    Public void run(){  
        System.out.println(this.getName() + " started");  
        Try {  
            Thread.sleep(2000);  
        }  
        catch(InterruptedException e){}  
        System.out.println(this.getName() + " finished");  
    }  
}
```

Contd...

1. By extending Thread class:

Example 2:

```
class MyThread extends Thread
{
    public void run(){
        for (int i = 0; i < 10 ;i++)
        {
            System.out.println("Child Thread");
        }
    }
}
```

Job of Thread

Defining a thread

Contd...

```
class ThreadDemo
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();//thread instantiation
        t.start();//starting of a thread
        for (int i = 0;i < 10 ;i++)
        {
            System.out.println("Main Thread");
        }
    }
}
```



Executed by
main thread

Case 1: Thread Scheduler

- It is part of JVM.
- It **is responsible to schedule threads** i.e. if multiple threads are waiting to get the chance of execution, then in which order threads will be executed is decided by thread scheduler.
- We cannot expect exact algorithm followed by thread scheduler it is varied from jvm to jvm. Hence we cannot expect threads execution order and exact output.
- Hence when ever situation comes multithreading there is no guarantee for exact output, but we can provide several possible outputs.

Case 2: difference b/w t.start() and t.run()

- In the case of t.start() a new thread will be created which is responsible for the execution of run() method.
- But in the case t.run() a new thread won't be created and run() method will be executed just like a normal method call by main thread.
- Hence in the above program if we replace t.start() with t.run() then the output is:

Child Thread
Child Thread

.

.

10 times
main Thread
main Thread

.

.

10 times

This total output
produced by only
main Thread

Case 3: Importance of Thread class start() method

- Thread class start() method is responsible to register the thread with thread scheduler and all other mandatory activities.
- Hence without executing thread class start() method there is no chance of starting a new thread in java due to this thread class start() method is considered as heart of multithreading.

Start()

{

1. Register this thread with thread scheduler
2. Perform all other mandatory activities
3. Invoke run() method

}

Case 4: Overloading of run() method

- Overloading of run() method is always possible, but Thread class start() method can invoke no-arg run() method the other overloaded method we have to call explicitly like a normal method call.
- Example:

```
class MyThread extends Thread
{
    public void run(){
        System.out.println("no-arg run");
    }
    public void run( int i){
        System.out.println("int-arg run");
    }
}
```

Overloaded
methods

Contd...

```
class Test
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();//thread instantiation
        t.start();//starting of a thread
    }
}
```

Output:

no-arg run

Case 5: If we are not Overriding run() method

- If we are not overriding run() method then **Thread** class run() method will be executed which has empty implementation, hence we won't get any output.

- **Example:**

```
class MyThread extends Thread
{
}
class Test
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
    }
}
```

No output

- Note: it is highly recommended to override run() method, other wise don't go for multithreading concept.

Case 6: Overriding of start() method

- If we overriding start() method, then our start() method will be executed just like a normal metho call and **new Thread** won't be created.

Example1:

Output:

Start method
Main method

Produced
only by main
Thread

```
class MyThread extends Thread
{
    public void start()
    {
        System.out.println("Start method");
    }
    public void run()
    {
        System.out.println("Run method");
    }
}
class Test2
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
        System.out.println("Main method");
    }
}
```

- **Note:** it is not recommended to override start() method other wise don't go for multithreading concept.

Case 6: Overriding of start() method

Example2:

```
class MyThread extends Thread
{
    public void start(){
        super.start();
        System.out.println("Start method");
    }
    public void run()
    {
        System.out.println("Run method");
    }
}
class Test2 {
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
        System.out.println("Main method");
    }
}
```

P-1 outputs:

Run metho
Start method
Main method

P-2 outputs:

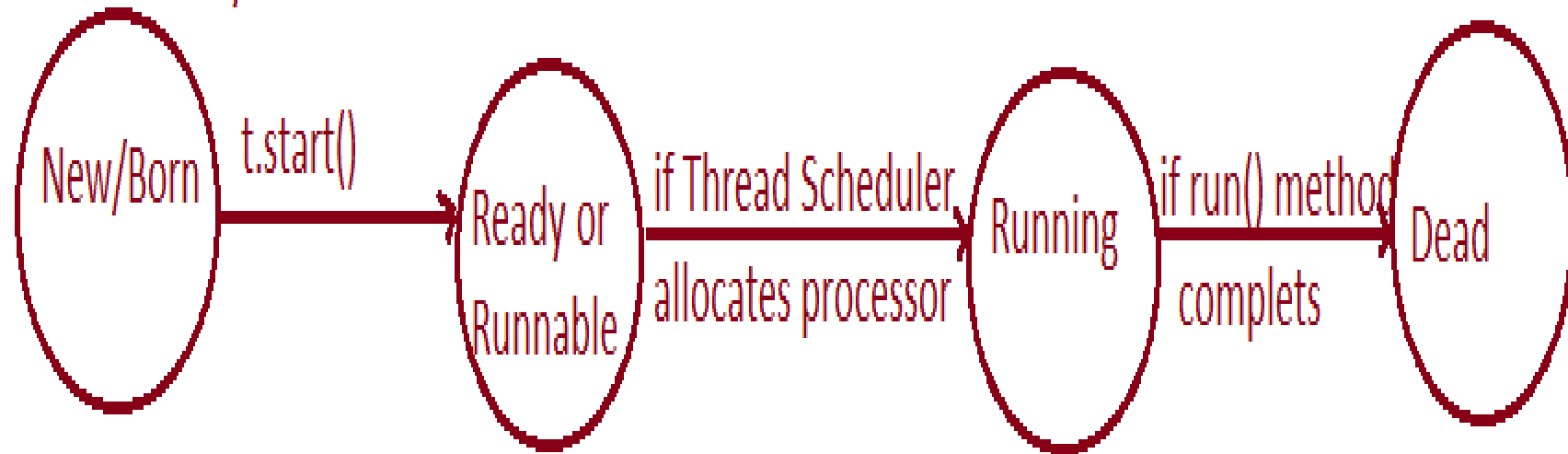
Start method
Run method
Main method

P-3 outputs:

Start method
Main method
Run method

Case 8: Thread Life Cycle

MyThread t= new MyThread



Case 9:

- After starting a Thread if we are trying to start the same Thread, then we will get runtime exception saying **IllegalThreadStateException**.
- **Example:**

```
class MyThread extends Thread
{
    public void run()
    {
        System.out.println("Run method");
    }
}
class Test3 {
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
        System.out.println("Main method");
        t.start();
    }
}
```

2. Defining a Thread by Implementing Runnable(I)

- We can define a Thread by implementing Runnable interface.
- Runnable interface present in 'java.lang' package and it contains one method run() method. Or

`public void run()`

Example1:

Defining
a Thread

```
class MyRunnable implements Runnable
{
    public void run()
    {
        for (int i = 0; i < 10; i++)
        {
            System.out.println("Child Thread");
        }
    }
}
```

Job of Thread executed by
child Thread

Contd...

```
class RunnableDemo1 {  
    public static void main(String[] args)  
    {  
        MyRunnable r = new MyRunnable();  
        Thread t = new Thread(r);  
        t.start();  
        for (int i = 0; i < 10; i++)  
        {  
            System.out.println("Main Thread");  
        }  
    }  
}
```

Target
Runnable

Executed by Main
Thread

Contd...

```
MyRunnable r = new MyRunnable();  
Thread t1 = new Thread();  
Thread t2 = new Thread(r);
```

- **Case 1:** `t1.start();`
 - A new Thread will be created and which is responsible for the execution of Thread class run() method, which has empty implementation.
- **Case 2:** `t1.run();`
 - No new Thread will be created and Thread class run() method will be executed just like a normal method call.
- **Case 3:** `t2.start();`
 - A new Thread will be created which is responsible for the execution of MyRunnable class run() method.
- **Case 4:** `t2.run();`
 - A new Thread will not be created and MyRunnable run() method will be executed just like a normal method call.

Contd...

- Case 5: `r.start();`
 - We will get compile time error saying MyRunnable class doesn't have start capability, **CE: cannot find symbol**
symbol method `start()`
location class `MyRunnable`
- Case 6: `r.run();`
 - No new Thread will be created and MyRunnable `run()` method will be executed like normal method call.

Thread class Constructors

```
1 Thread t = new Thread();  
2 Thread t = new Thread(Runnable r);  
3 Thread t = new Thread(String s);  
4 Thread t = new Thread(Runnable r, String s);  
5 Thread t = new Thread(ThreadGroup g, String s);  
6 Thread t = new Thread(ThreadGroup g, Runnable r);  
7 Thread t = new Thread(ThreadGroup g, String s);  
8 Thread t = new Thread(ThreadGroup g, Runnable r, String s);  
9 Thread t = new Thread(ThreadGroup g, Runnable r, String s, long stackSize);
```

Thread Priorities

- Every Thread In java has some priority it may be default priority generated by jvm or customized priority provided by programmer.
- The valid range of Thread priorities is 1 to 10. where 1 is min priority and 10 is max priority.
- Thread class defines the following constants to represent some standard priorities:
 - ✓ Thread.MIN_PRIORITY ----- 1
 - ✓ Thread.NORM_PRIORITY ----- 5
 - ✓ Thread.MAX_PRIORITY -----10

Contd...

- Thread scheduler will use priorities while allocating processor.
- The Thread which is having highest priority will get the chance first.
- If two Threads having same priority then we cannot expect exact execution order it depends on Thread scheduler.
- Thread class defines the following methods to get and set priority of a thread:

```
public final int getPriority()
```

```
public final void setPriority(int p)
```



Allowed values
range: 1 to 10

Contd...

- Other wise we will get runtime exception saying:
IllegalArgumentException.

Example: t.setPriority(7)-- valid

t.setPriority(17)-- **RE: IllegalArgumentException**

❑ Default priority:

- The default priority only for the main thread is 5. but for all remaining threads default priority will be inherited from parent to child i.e. what ever priority parent thread has the same priority will be there for the child thread.

Contd...

Example 1:

```
class MyThraed extends Thread
{
}
class Test4{
    public static void main(String[] args)
    {
        System.out.println(Thread.currentThread().getPriority());
        Thread.currentThread().setPriority(15); //RE:IllegalArgumentException
        Thread.currentThread().setPriority(7);
        MyThread t = new MyThread();
        System.out.println(t.getPriority()); —————> 7
    }
}
```

Contd...

Example 2:

```
class MyThread extends Thread
{
    public void run()
    {
        for (int i = 0; i < 10; i++)
        {
            System.out.println("Child Thread");
        }
    }
}
class ThreadPriority {
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.setPriority(10);
        t.start();
        for (int i = 0; i < 10; i++)
        {
            System.out.println("Main Thread");
        }
    }
}
```

Contd...

- The output for the above program is:

Child Thread

Child Thread

...

10 time

Main Thread

Main Thread

...

10 times

- **Note:** some platforms will not provide proper support for Thread priorities.

Preventing Thread from Execution

- We can prevent a Thread execution by using the following methods:

1. `yield()`
2. `join()`
3. `sleep()`

1. `yield()`:

- `Yield()` method causes to pause current executing Thread to give the chance for waiting threads of same priority.
- If there is no waiting thread or all waiting threads have low priority then same thread can continue its execution.
- If multiple threads are waiting with the same priority, then which waiting thread will get the chance depends on thread scheduler.

Contd...

- The thread which is yielded, get the chance of resuming execution once again depends on thread scheduler.
- The complete prototype of yield method is:

`public static native void yield();`

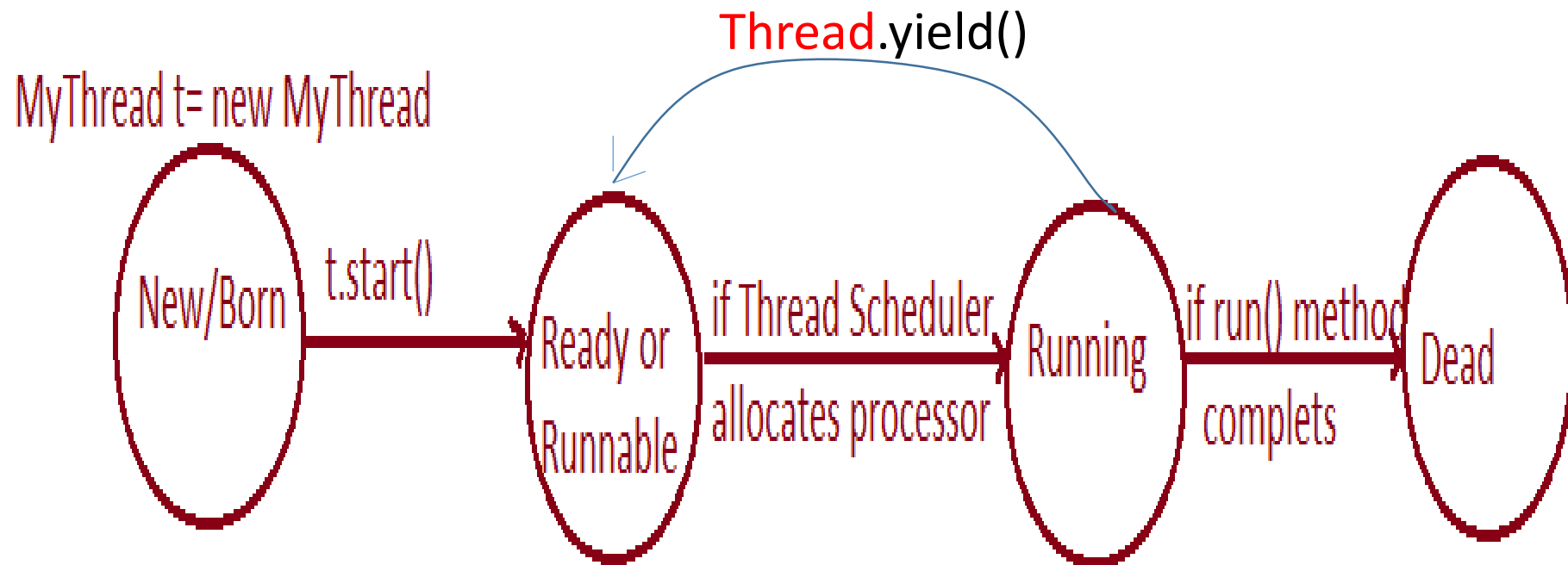


Fig: Thread transition for yield() method call.

Contd...

Example:

```
class MyThread extends Thread
{
    public void run()
    {
        for (int i = 0; i < 10; i++)
        {
            System.out.println("Child Thread");
            Thread.yield();
        }
    }
}

class ThreadYieldDemo
{
    public static void main(String[] args)
    {
        MyThread t = new MyThread();
        t.start();
        for (int i = 0; i < 10; i++)
        {
            System.out.println("Main Thread");
        }
    }
}
```

Contd...

- In the above program if we are commenting “**Thread.yield();**” then both the threads will be executed simultaneously and we cannot expect which thread will complete first.
- If we are not commenting “**Thread.yield();**” then child thread always call yield() method, because of that main Thread will get the chance more number of times and the chance of completing main thread first is high.

2. join():

- If a thread wants to wait until completing some other thread then we should go for join() method.
- For example if a thread t1 wants to wait until completing t2 then t1 has to call “**t2.join();**”.
- If t1 executes **t2.join()** then immediately t1 will be entered into waiting state until t2 completes.
- Once t2 completes then t1 can continue its execution.

Contd...

Example:

**Venue fixing
(t1)**
⚡

**Weeding cards
printing
(t2)**
⚡
t1.join();

**Weeding
cards
distribution
(t3)**
⚡
t2.join();

- Weeding cards printing Thread (t2) has to wait until venue fixing thread (t1) completion, hence t2 has to call t1.join();.
- Weeding cards distribution thread (t3) has to wait until weeding cards printing thread (t2) completion hence t3 has to call t2.join();.

Contd...

- The complete signature of join() methods are:
 - `public final void join(); throws InterruptedException`
 - `public final void join(long ms); throws InterruptedException`
 - `public final void join(long ms, int ns); throws InterruptedException`
- **Note:** every join() method **throws InterruptedException** which is checked exception.
- Hence it is compulsory to handle it either by using **try--- catch** or by **throws key-word** other wise we will get compile time error.

Contd...

- aa

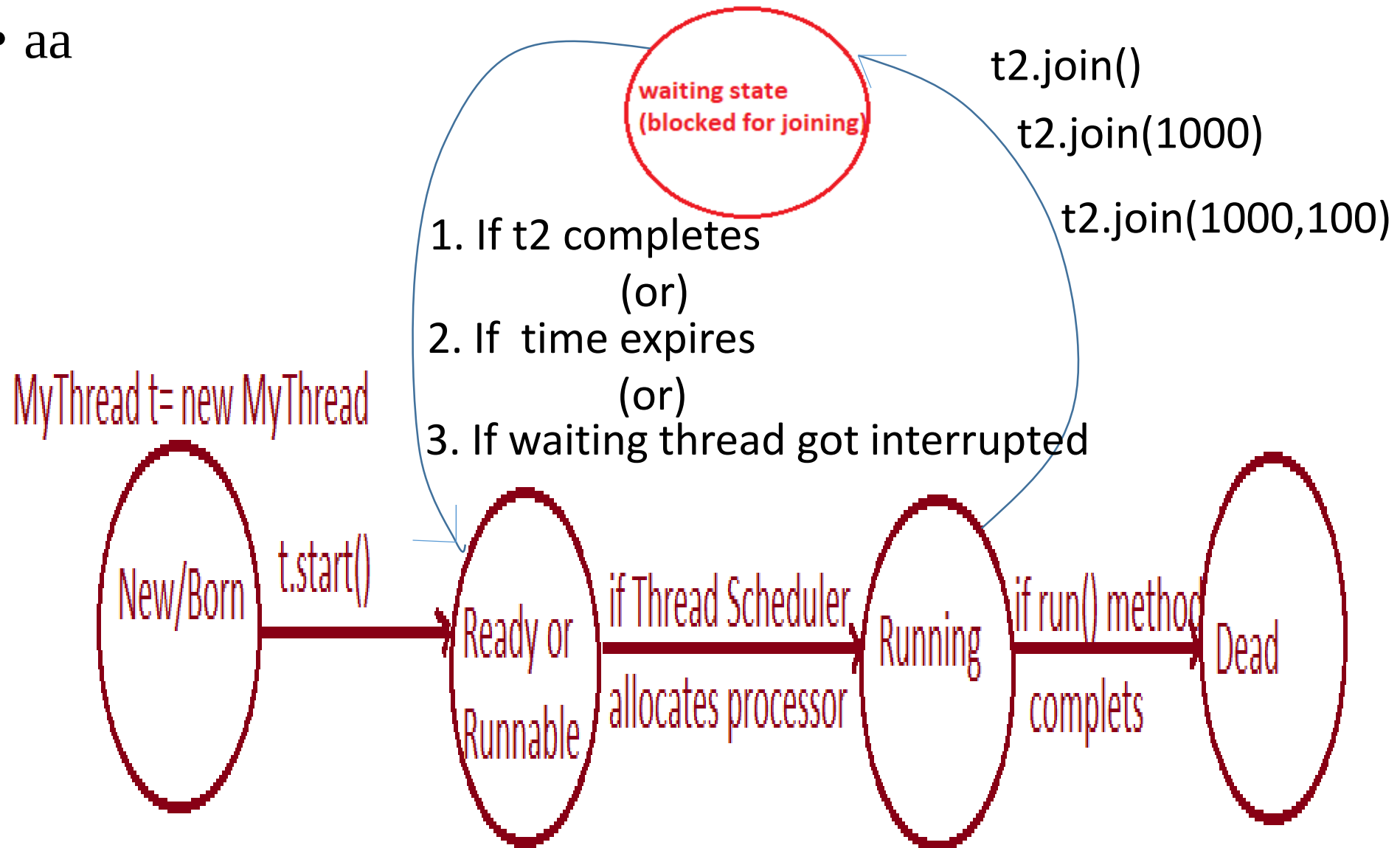


Fig: The impact of `join()` method on thread life cycle

Case 1: waiting of Main Thread until Completing Child Thread

Example 1:

```
class MyThread extends Thread
{
    public void run() {
        for (int i = 0; i < 10; i++) {
            System.out.println("Child Thread");
            try{
                Thread.sleep(2000); }
            catch (InterruptedException e) {
            }
        }
    }
}

class ThreadJoinDemo{
    public static void main(String[] args) throws InterruptedException {
        MyThread t = new MyThread();
        t.start();
        t.join();
        for (int i = 0; i < 10; i++){
            System.out.println("Main Thread");
        }
    }
}
```


Contd...

- If we comment “`t.join();`” then both main and child threads will be executed simultaneously and we cannot expect exact output.
- If we are not commenting “`t.join();`” then main thread calls `join()` method on child thread object.
- Hence main thread will wait until completing child thread. In this case output is:

Child Thread

Child Thread

...

10 times

Main Thread

Main Thread

...

10 times

Case 2: waiting of Child Thread until Completing Main Thread

Example 2: `class MyThread extends Thread`

```
{
    static Thread mt;
    public void run() {
        try {
            mt.join();
        }
        catch (InterruptedException e){
        }
        for (int i = 0; i < 10; i++) {
            System.out.println("Child Thread");
        }
    }
}

class ThreadJoinDemo1{
    public static void main(String[] args) throws InterruptedException {
        MyThread.mt = Thread.currentThread();
        MyThread t = new MyThread();
        t.start();
        for (int i = 0; i < 10; i++){
            System.out.println("Main Thread");
            Thread.sleep(2000);
        }
    }
}
```

Contd...

- In the above example child thread calls `join()` on main thread object. hence child thread has to wait until completing main thread.
- In this case output is:

Main Thread

Main Thread

...

10 times

Child Thread

Child Thread

...

10 times

Case 3: Deadlock

- If main thread calls `join()` method on child thread object and child thread calls `join()` method on main thread object then **both threads will wait each other for ever** and the program will be stacked or paused (this is something like `Deadlock`).

Contd...

➤ Example:

```
class MyThread extends Thread
{
    static Thread mt;
    public void run() {
        try {
            mt.join();
        }
        catch (InterruptedException e){
        }
        for (int i = 0; i< 10; i++) {
            System.out.println("Child Thread");
        }
    }
}

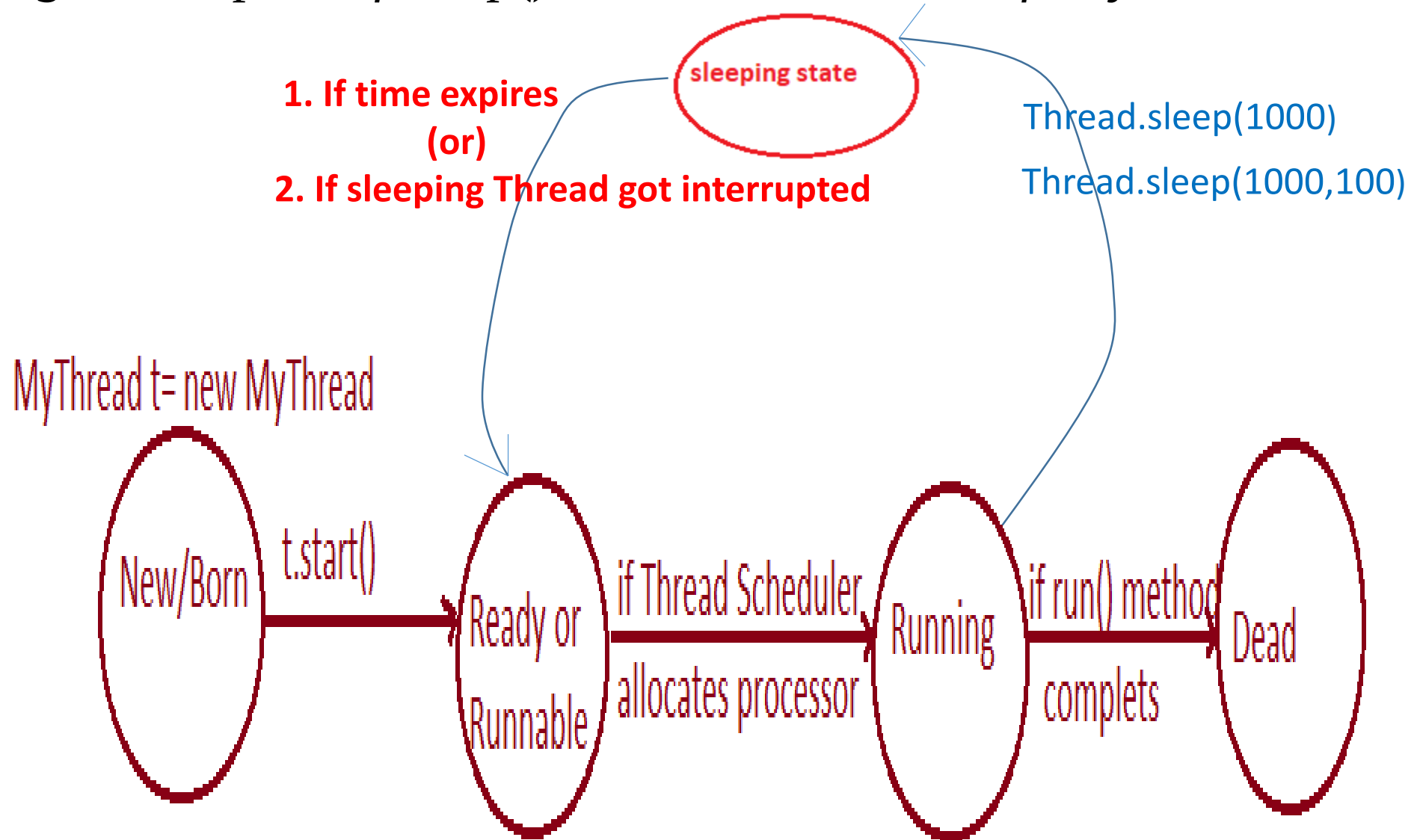
class ThreadJoinDemo1{
    public static void main(String[] args) throws InterruptedException {
        MyThread.mt = Thread.currentThread();
        MyThread t = new MyThread();
        t.start();
        t.join();
        for (int i = 0; i<10; i++){
            System.out.println("Main Thread");
            Thread.sleep(2000);
        }
    }
}
```

3. sleep() method:

- If a thread don't want to perform any operation for a particular amount of time then we should go for sleep() method.
- Method signature of sleep() method:
 - `public static native void sleep(long ms) throws InterruptedException`
 - `public static void sleep(long ms, int ns) throws InterruptedException`
- **Note:** every `sleep()` method throws `InterruptedException`, which is checked exception.
- Hence when ever we are using sleep() method it is compulsory that we should handle `InterruptedException` either by `try --- catch` or by `throws` key-word other wise we will get compile time error.

Contd...

Fig: the impact of sleep() method on Thread life Cycle



Contd...

Example:

```
class SlideRotator
{
    public static void main(String[] args) throws InterruptedException
    {
        for (int i = 1; i<=10; i++)
        {
            System.out.println("Slide-" +i);
            Thread.sleep(5000);
        }
    }
}
```


How a Thread can Interrupt another Thread?

- A thread can interrupt a sleeping thread or waiting thread by using interrupt() method of **Thread** class.

`public void interrupt()`

Example:

```
class MyThread extends Thread
{
    public void run() {
        try {
            for (int i = 0; i < 10; i++) {
                System.out.println("I'm Lazy Thread!");
                Thread.sleep(2000);
            }
        } catch (InterruptedException e){
            System.out.println("I got Interrupted!");
        }
    }
}

class ThreadInterruptedDemo {
    public static void main(String[] args) throws InterruptedException {
        MyThread t = new MyThread();
        t.start();
        t.interrupt();
        System.out.println("End of Main Thread");
    }
}
```

Contd..

- If we comment “`t.interrupt();`” in the above program, then main thread will not interrupt child thread.
- In this case child thread will execute for-loop 10 times.
- If we are not commenting “`t.interrupt();`” then main thread interrupts child thread.
- In this case output is:

End of Main Thread
I'm Lazy Thread!
I got Interrupted!

Comparison table of yield(), join(), and sleep() methods

property	Yield()	Join()	Sleep()
1. purpose?	If a thread wants to pause its execution to give the chance for remaining threads of same priority, then we should go for yield() method.	If a thread wants to wait until completing some other thread, then we should go for join() method.	If a thread don't want to perform any operation for a particular amount of time, then we should go for sleep() method.
2. Is it overloaded?	No	Yes	Yes
3. Is it final?	No	Yes	No
4. Is it throws InterruptedException ?	No	Yes	Yes
5. Is it native?	Yes	No	- Sleep(long ms) is native but - Sleep(long ms, int ns) is non-native

Synchronization

- **Synchronized** is the modifier applicable only for **methods and blocks** but not classes and variables.
- If multiple threads are try to operate simultaneously on the same java object then there may be a chance of **data inconsistency problem**.
- To overcome this problem we should go for **synchronized** key-word.
- If a **method or block declared as synchronized** then at **a time only one thread is allowed to execute that method or block** on the given object. So that data inconsistency problem will be resolved.
- The main advantage of synchronized key-word is we can resolve data inconsistency problems. But the main disadvantage of synchronized key-word is it increases waiting time of threads and creates performance problems. Hence if there is no specific requirement then it is not recommended to use synchronized key-word.

Contd...

- Internally synchronization concept is implemented by using lock.
- Every object in java has a unique lock.
- When ever we are using synchronized key-word then only lock concept come into the picture.
- If a thread wants to execute synchronized method on the given object first it has to get lock of that object.
- Once thread got the lock then it is allowed to execute any synchronized method on that object.
- Once method execution completes automatically thread releases the lock.

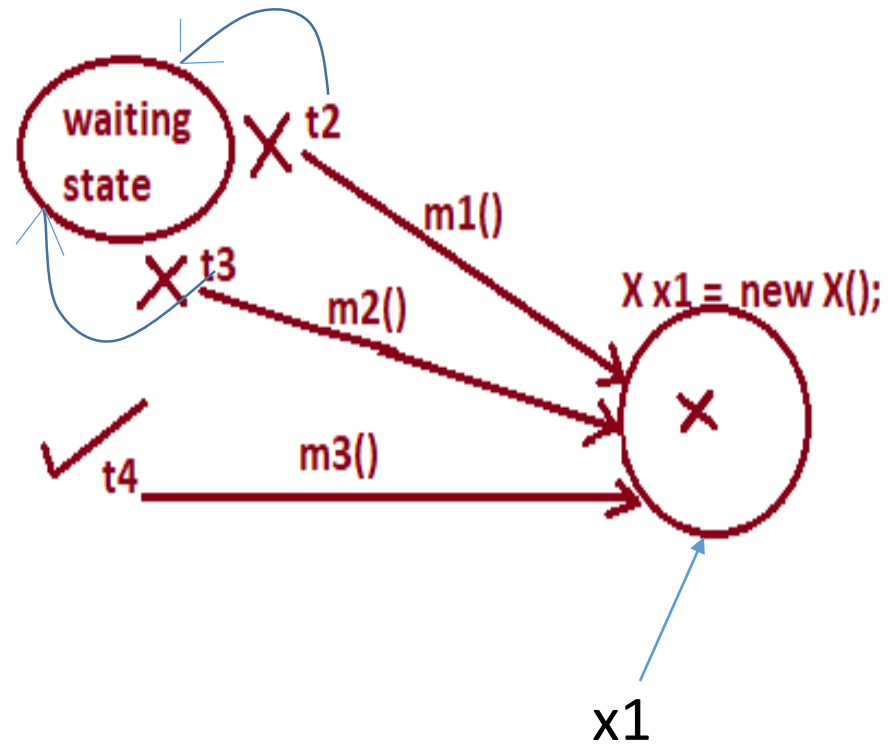
Contd...

- Acquiring and releasing lock internally takes care by JVM and programmer not responsible for this activity.
- While a thread executing synchronized method on the given object the remaining threads are not allowed to execute any synchronized method simultaneously on the same object.
- But remaining threads are allowed to execute non-synchronized methods simultaneously.

Example:

```
class X
{
    synch m1()
    synch m2()
        m3()
}
```

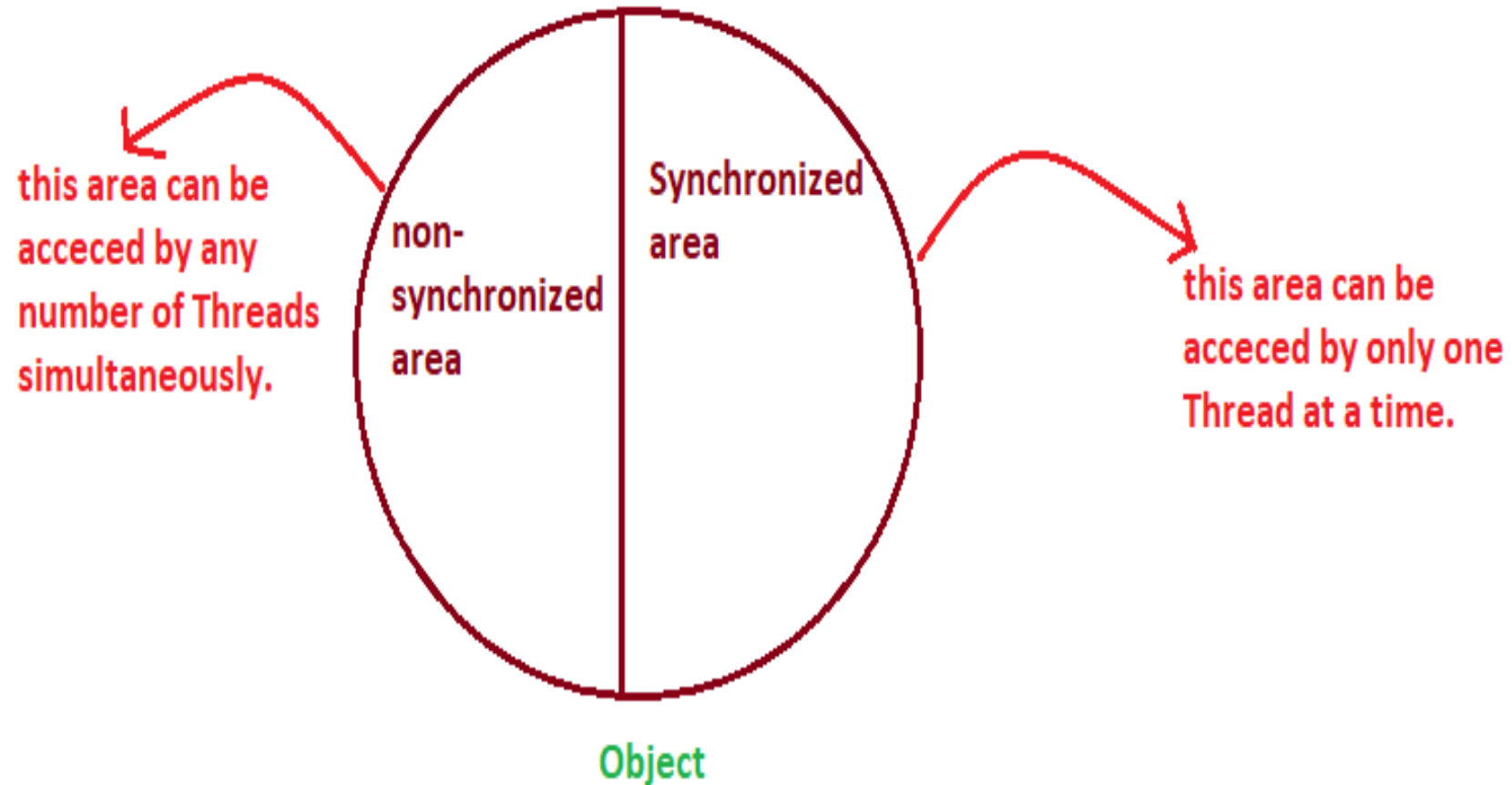
Contd...



```
t1 → L (X)
↓
m1()
{
  ---
  ---
  ---
}
```

Contd...

- Lock concept is implemented based on object but not based on method.



Contd...

Example:

Class X

{

synchronized method

{

**where ever we are performing update operation
[Add/delete/insert/update]
i.e. where state of object changed**

}

Non-synchronized method

{

**where ever state of object won't be changed
like read() operation**

}

}

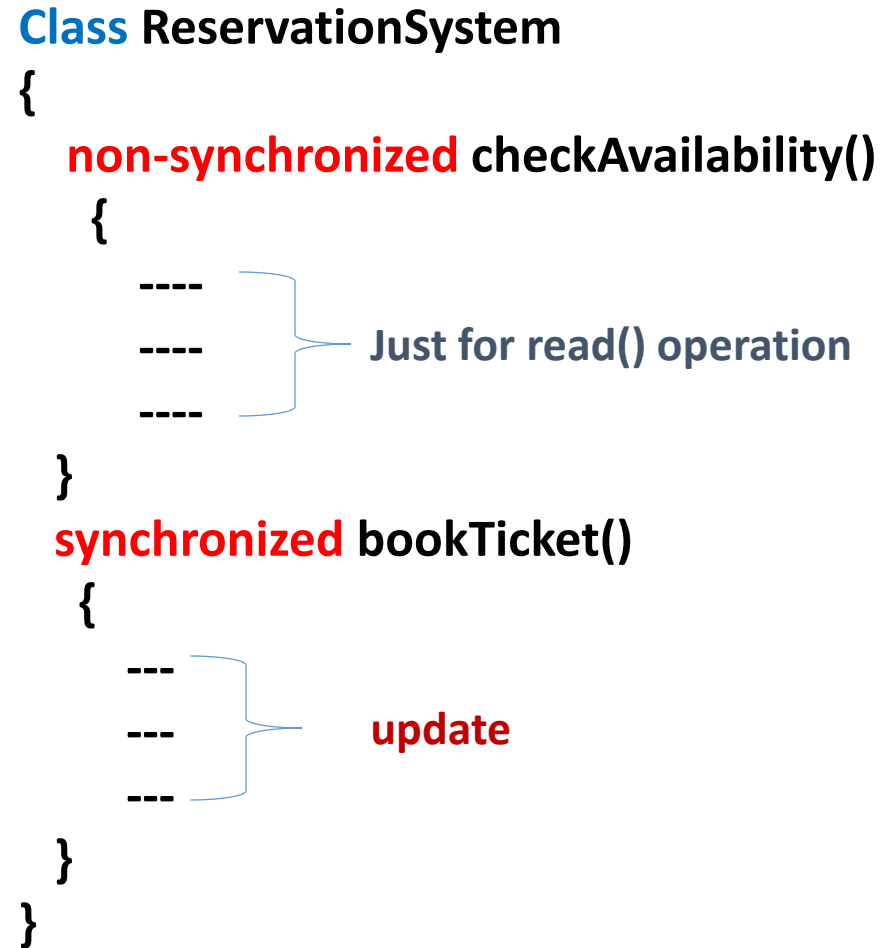
Contd...

Example:

```
Class ReservationSystem
{
    non-synchronized checkAvailability()
    {
        ----
        ----
        ----
    }
    synchronized bookTicket()
    {
        ----
        ----
        ----
    }
}
```

Just for read() operation

update

The diagram illustrates two methods within the ReservationSystem class. The first method, checkAvailability(), is marked as 'non-synchronized' and contains three lines of code represented by dashed lines. A blue bracket groups these lines with the annotation 'Just for read() operation'. The second method, bookTicket(), is marked as 'synchronized' and also contains three lines of code represented by dashed lines. A blue bracket groups these lines with the annotation 'update'.

Contd...

Example 1:

```
class Display
{
    public synchronized void wish(String name)
    {
        for (int i = 0; i<10; i++)
        {
            System.out.print("Good Morning:");
            try
            {
                Thread.sleep(2000);
            }
            catch (InterruptedException e){}
            System.out.println(name);
        }
    }
}
```

Contd...

```
class MyThread extends Thread
{
    Display d;
    String name;
    MyThread(Display d, String name)
    {
        this.d = d;
        this.name = name;
    }
    public void run()
    {
        d.wish(name);
    }
}
```

Contd...

```
class SynchronizedDemo
{
    public static void main(String[] args)
    {
        Display d = new Display();
        MyThread t1 = new MyThread(d, "Alemu");
        MyThread t2 = new MyThread(d, "Dani");
        MyThread t3 = new MyThread(d, "Kebede");
        MyThread t4 = new MyThread(d, "Hiwot");
        t1.start();
        t2.start();
        t3.start();
        t4.start();
    }
}
```

End of Chapter

Thank you !